

FORMA NORMALE CHOMSKY:

- ① Elimino Produzioni che non mi portano a Terminali ($|V|$ Passi max.)
- ② Elimino Produzioni non Raggiungibili da S ($|V|+|T|$ Passi Max.)
- ③ Elimino ϵ -Produzioni, Quelle che mi portano SOLO ad ϵ ($|V|$ Passi)
- ④ Eliminazione Produzioni UNITARIE

Mi fa Arrivare ad un Albero di derivazione Binario nel Quale so che ho un Altezza $n+1$ e 2^n foglie (Simboli Terminali)

DIMOSTRO CHE È REGOLARE

Se il linguaggio Generato è finito lo è Implicitamente, Quindi $\emptyset$ lo è

① Creare Automa e dare un Significato ad Ogni Stato.

INIZIO LA DIMOSTRAZIONE:

② Scelgo lunghezza Per la BASE che mi Permetta di Generare sia Stringhe $\in L$ che non e Raggiunga ogni Stato

NB! Ricordarsi di Specificare Anche Cosa Succede a Quelle Prima con le seguenti Ipotesi Induttive:

Si Assume Per $|x| \leq n$ Vale $x \in L \Rightarrow \hat{\delta}(q_0, x) \in F$

Si Assume Per $|x| \leq n$ Vale $x \notin L \Rightarrow \hat{\delta}(q_0, x) \notin F$

Vado a dimostrare tutti i Possibili Casi di Stringhe $|x| = n+1$

③ Devo dimostrare Per Induzione Su $|x|$ che $(L = L(M))$

$x \in L \Rightarrow \hat{\delta}(q_0, x) = q_i \in F$

$x \notin L \Rightarrow \hat{\delta}(q_0, x) = q_i \notin F$

Con l'obiettivo $x \in L$ e $x \notin L$ con Stringhe $|x| = n+1$

Con tutte le Possibili Versioni di x' che Rispettano il caso SPECIFICO

Chiuso Rispetto a UNIONE-CONCATENAZIONE-KLEENE-INTERSEZ.-COMPLEMENTO

DIMOSTRO CHE NON È REGOLARE $\{0^n 1^n \mid n \in \mathbb{N}\}$

Sfrutto la Negazione del PUMPING LEMMA, Quindi:

$\forall n \in \mathbb{N}. \exists z \in L. |z| \geq n. \exists u, v, z \in \Sigma^*$ tale che:

1. $z = uvw$

2. $|uv| \leq n$

3. $|v| > 0$

4. $\exists i > 0. uv^i w \notin L$

Devo riuscire a RIDURMI ad un ASSURDO Su una delle CONDIZIONI

Devo CONCLUDERE con $\exists i > 0. uv^i w \notin L$ Quindi $L \notin REG.$

Contare Indefinitamente / Richiedo Memoria per Salvare INFO

DIMOSTRO CF:

Sono COPPIE A SPECCHIO, Identifico delle Relazioni

Dimostro $L = L(G)$ Quindi:

① $x \in L \Rightarrow S \xRightarrow{G}_* x$ per Induzione su $|x|$

② $S \xRightarrow{G}_i x \Rightarrow x \in L$ Per Induzione su i (len. derivazione)

① **BASE:** Stringa più corta $\in L$ e Mostro derivazione valida

I.I: $\forall x \in L, |x| \leq n$ esiste $S \xRightarrow{*}_* x$ e devo dim. per $|x| = n+1$

PASSO: Riscrivo x in Termini di x' in modo che applico I.I su x' e dalla $S \xRightarrow{*}_* x'$ devo ricostruire la derivazione per x ragionando su G

② **BASE:** Prendo Derivazione di Simboli TERMINALI più corta $\in G$ e verifico che Stringa Generata $\in L$.

I.I: Se $S \xRightarrow{i}_i x$ allora $x \in L$ e devo dimostrare $S \xRightarrow{i+1}_{i+1} x, x \in L$

PASSO: Suppongo di Avere $S \xRightarrow{i+1}_{i+1} x$ e Ragionando su G devo Arrivare alla derivazione Lunga i Per Applicare I.I.

Chiuso Rispetto a UNIONE-CONCATENAZIONE-CHiusura di KLEENE Ma NON rispetto all' INTERSEZIONE e COMPLEMENTO.

DIMOSTRO NON CF: $\{a^n b^n c^n \mid n \in \mathbb{N}\}$

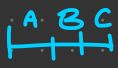
$\forall k \in \mathbb{N} (k \neq 0), \exists z \in L. |z| \geq k$ e $\forall uvwxy = z$ dove $|vwx| \leq k$ e $|vx| > 0$

So che $\exists i \in \mathbb{N}. uv^i wx^i y \notin L$

Ricordarsi che su K NON ci vanno dei vincoli Quindi devo prendere un $K \in L$ Senza vincoli speciali

Se devo CONTARE Più di 2 dipendenze Interconnesse. Un vincolo Tra 2 Gruppi Riesco ma NON di più, Oppure ho una TUPLA

$$\#Casi = (\#Sotto_Parti \cdot 4) - 3$$

 Se a Cavallo Tra $A \in B$ e Ripeto Stringer Cambio Strutt. Str

$L \text{ CF} \Rightarrow \exists k \in \mathbb{N}. \forall z \in L$ dove $|z| \geq k. \exists uvwxy = z$ dove $|vx| > 0$ e $|vwx| \leq k$ so che $\forall i \in \mathbb{N} uv^i wx^i y \in L$

Supponendo G in forma Normale di Chomsky $\forall z \in L$ l'Albero di derivazione Binario con Altezza 2^h e 2^{h-1} foglie.

TEORIA:

BASI - LINGUAGGI FORMALI:

SIMBOLO $\Rightarrow$ Entità Primitiva del Linguaggio (MATTONCINO BASE)

PAROLA / STRINGA $\Rightarrow$ Sequenze di Simboli

LUNGHEZZA $\Rightarrow$ # Occorrenze di Simboli ($\neq$ # Simboli)

ALFABETO $\Rightarrow$ Collezione finita di Simboli su cui Costruiamo il linguaggio

Σ^* è il linguaggio di TUTTE le possibili Stringhe su $\Sigma \neq \emptyset$.
È infinito Numerabile $\rightarrow \exists$ una Corrispondenza Tra Σ^* e $\mathbb{N}$

PROBLEMA INFORMATICO e LIMITI ANNESSI:

TEOREMA CANTOR $\Rightarrow |S| < |P(S)|$
 $\swarrow$ Parti di S $\rightarrow$ Sottoinsiemi di S
 $\searrow$ Cardinalità $2^{|S|}$

Proprio Per CANTOR posso Affermare che $|\mathbb{N}| < |P(\mathbb{N})|$
 $\swarrow$ Mondo Discreto $\nwarrow$ Mondo Continuo

PROBLEMA $\Rightarrow f: \mathbb{N} \rightarrow \mathbb{N} \equiv f \subseteq \mathbb{N} \times \mathbb{N}$. Come Insieme di COPPIE su $\mathbb{N}$ che è un Sottoinsieme di $\mathbb{N}$ Numerabile $\Rightarrow |f| \leq |\mathbb{N}|$

PROGRAMMA $\Rightarrow$ Seq. Simboli finita in un Alfabeto (Chiamate Istruzioni)

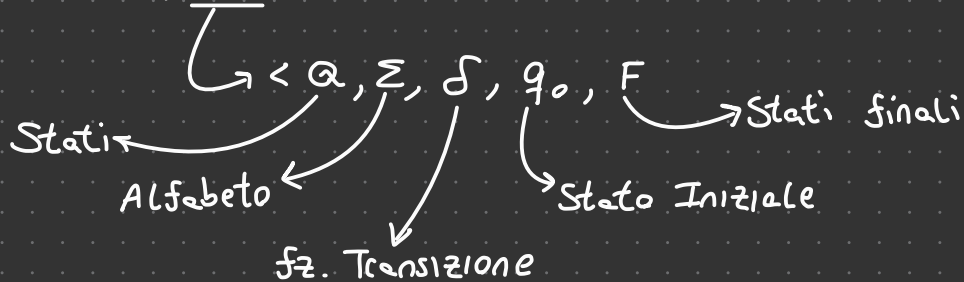
INSIEME PROGRAMMI $\subseteq \Sigma^*$ e So che INSIEME PROGRAMMI è Numerabile visto che lo è Σ^* e Quindi:

$$\# |\text{INSIEME PROGRAMMI}| = |\mathbb{N}|$$

Il #fz descrivibili come Algoritmi è Strettamente Inferiore al #fz.

LINGUAGGI REGOLARI:

Riconosciuti da un DFA



$$\hat{\delta}: Q \times \Sigma^* \rightarrow Q$$

$$\begin{cases} \hat{\delta}(q, \varepsilon) = q \\ \hat{\delta}(q, wa) = \delta(\underbrace{\hat{\delta}(q, w)}_{\text{RAGGIUNGO STATO } q'}, a) \end{cases}$$

L è Regolare se $\exists$ DFA M . $L = L(M) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \in F\}$

↑
Uguaglianza da DIMOSTRARE per INDUZIONE

$\varepsilon \in L$ è un Ragionamento a Parte, Nel Caso in cui fosse Ambiguo Specificare come lo ho Interpretato.

DIMOSTRARE CHE È UN LINGUAGGIO REGOLARE: Automa M e Voglio dimostrare che $L = L(M)$ Per Induzione su lunghezza di x

Significa: $L \subseteq L(M) \wedge L(M) \subseteq L$

$$x \in L \Rightarrow x \in L(M)$$

$\wedge$

$$x \in L(M) \Rightarrow x \in L$$

$$x \in L \Rightarrow \hat{\delta}(q_0, x) \in F$$

$\wedge$

$$\hat{\delta}(q_0, x) \in F \Rightarrow x \in L$$

Per Semplicità Sfrutto la Sua CONTROINVARIANTE

$$x \notin L \Rightarrow \hat{\delta}(q_0, x) \notin F$$

Quindi deve DIMOSTRARE:

$$\begin{cases} x \in L \Rightarrow \hat{\delta}(q_0, x) \in F \Leftrightarrow L \subseteq L(M) \\ x \notin L \Rightarrow \hat{\delta}(q_0, x) \notin F \Leftrightarrow L(M) \subseteq L \end{cases}$$

Dare un Ruolo ad ogni Stato, aiuta poi Per la DIMOSTRAZIONE

Che Avverrà Contemporaneamente per ENTRAMBE e Quindi la BASE deve Contenere L .

① Devo Trovare una **BASE** con lunghezza Minima della Stringa che mi **Permetta di Identificare Sia Stringhe $\in L$ che $\notin L$** .

② **IPOTESI INDUTTIVA** $\Rightarrow$ Supporre che $\forall$ Stringhe che Vanno oltre la Base di lunghezza $\leq n$ vale che:

$$(*) \begin{cases} x \in L \Rightarrow \hat{\delta}(q_0, x) \in F \\ x \notin L \Rightarrow \hat{\delta}(q_0, x) \notin F \end{cases}$$

③ **PASSO INDUTTIVO** $\Rightarrow$ Se x di lunghezza $+1$ allora Vale $(*)$ ed Avrò: $x = x'a$

$\in \Sigma^*$ $\leftarrow$ $\xrightarrow{\quad}$ $\in \Sigma^*$

Prendo $x \in L$ (o $x \notin L$), Quindi Caratterizzo x in funzione di L

LINGUAGGIO L

$$x = x'a$$

$|x'| = n$ e Posso Applicare I.I.

Automa M

Dagli Stati finali Raggiunti con x' leggo **a** e mostro dove si Arriva:

- in F se $x \in L$

- Non in F se $x \notin L$

$$\text{Se } x' \in L \Rightarrow \hat{\delta}(q_0, x') \in F$$

$$\text{Se } x' \notin L \Rightarrow \hat{\delta}(q_0, x') \notin F$$

Solo Con **IPOTESI INDUTTIVA** ignoro Linguaggio e uso Automa

NFA: Non determinismo NON mi Aumenta il Potere Espressivo

Sempre: $\langle Q, \Sigma, \delta, q_0, F \rangle$

Stati $\leftarrow$ Q
Alfabeto $\leftarrow$ Σ
Stati finali $\leftarrow$ F
Stato Iniziale $\leftarrow$ q_0

Prende uno Stato ed un simbolo e Restituisce

L'Insieme degli Stati Raggiungibili $\Rightarrow$ f.z. non Determ.

Cambiando δ Varia Anche: $\hat{\delta}: Q \times \Sigma^* \rightarrow P(Q)$ definita Come:

$$\begin{cases} \hat{\delta}(q, \varepsilon) = \{q\} \\ \hat{\delta}(q, xa) = \bigcup_{q' \in \hat{\delta}(q, x)} \delta(q', a) \end{cases}$$

Insieme con un Solo STATO.
[Collezione tutto Quello che Posso Raggiungere]

x è Accettata da N (NFA) se $\hat{\delta}(q_0, x) \cap F \neq \emptyset$, Quindi se esiste in N almeno un Cammino che leggendo x Porta ad uno Stato finale.

Non DETERMINISMO $\rightarrow$ Mi Basta Solo 1 Caso per Essere Accettato
 $\rightarrow$ ho più Archi con Stesso Simbolo dal un nodo.

Non determinismo è un CASO PARTICOLARE di DETERMINISMO che di solito mi Semplifica la Rappresentazione Grafica. (DO PIÙ SIGNIFICATO AD UN SOLO NODO, DIPENDE DAL PERCORSO CHE SEGUE).

$M \text{ è DFA} \Rightarrow M \text{ è anche NFA}$

$$\delta(q, a) = q' \Rightarrow \delta(q, a) = \{q'\}$$

TEOREMA RABIN-SCOTT: M è un NFA, Allora $\exists M'$ DFA che riconosce lo stesso Linguaggio $\rightarrow L(M) = L(M')$

✓ Costruisco M' :

$Q' = P(Q)$ e Abbiamo una corrispondenza Biunivoca Tra $P \subseteq Q$ e Q'

$$q_0 = \{q_0\} \in P(Q) = Q'$$

$$F' = \{P \subseteq Q \mid P \cap F \neq \emptyset\}$$

$$\delta': Q' \times \Sigma \rightarrow Q' \quad \delta'(P, a) = \bigcup_{q \in P} \underbrace{\delta(q, a)}_{\subseteq Q} \in Q' = P(Q)$$

ϵ -NFA: Mi Posso spostare Tra Stati Senza leggere Nessun

Simbolo (TRANSIZIONI CHE ESPLICITO CON ϵ)



ϵ -NFA $N = \langle Q, \Sigma, \delta, q_0, F \rangle$ dove $\delta: Q: (\Sigma \times \{\epsilon\}) \rightarrow P(Q)$

Insieme perché è non deterministico

Devo Ridefinire $\hat{\delta}$, Ma prima la **ϵ -Closure** ovvero **Tutti gli Stati che posso Raggiungere facendo le ϵ -Transizioni:**

$$\epsilon\text{-Closure}(P) = \bigcup_{q \in P} \epsilon\text{-Closure}(q)$$

$$\begin{cases} \hat{\delta}(q, \varepsilon) = \varepsilon\text{-Closure}(q) & (\text{Posso Andare in Altri Stati}) \\ \hat{\delta}(q, xa) = \bigcup_{q' \in \hat{\delta}(q, x)} \varepsilon\text{-Closure}(\delta(q', a)) \end{cases}$$

$$L(M) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \cap F \neq \emptyset\}$$

OPERAZIONI SUI LINGUAGGI REGOLARI:

Concatenazione $\Rightarrow L_1 \cdot L_2 = \{xy \in \Sigma^* \mid x \in L_1 \wedge y \in L_2\}$

Chiusura di Kleene $\Rightarrow L^* = \bigcup_{i \in \mathbb{N}} L^i$ e se L^+ allora L non è Inclusa.

Unione $\Rightarrow L_1 \cup L_2 = \{x \in \Sigma^* \mid x \in L_1 \vee x \in L_2\}$

ESPRESSIONI REGOLARI:

$\emptyset$ (L Senza Stringhe), ε ed un Simbolo a Sono Espressioni Regolari.

Se r e s Sono Espressioni Regolari allora:

$r + s$

$r \cdot s$

r^* / r^+

Sono Espressioni Regolari che Rappresentano Ognuna un linguaggio

CONCATENAZIONE: $L_1 \cdot L_2 = \{xy \in Z^* \mid x \in L_1 \wedge x \in L_2\}$

CHIUSURA DI KLEENE: $L^* = \bigcup_{i \in \mathbb{N}} L^i$ Ovvero Ripetere un certo numero di volte le stringhe di L .

Con $L^+ = \bigcup_{i > 0} L^i$ dove ε NON è incluso.

PUMPING LEMMA: Strumento Per dimostrare che NON è regolare essendo una CONDIZIONE NECESSARIA per la REGolarità

Se L linguaggio Regolare allora $\exists k \in \mathbb{N}$. $\forall$ Stringa $z \in L$
 $|z| \geq k$ esiste una suddivisione di z in 3 parti u, v, w tali
che $z = uvw$ e $|uv| \leq k$ con $|v| > 0$ tale che $\forall i \in \mathbb{N}$
 $uv^i w \in L$.

L Regolare $\Rightarrow$ Vale proprietà descritta



Non Vale proprietà descritta $\Rightarrow L$ NON è Regolare.

z per Essere Riconosciuto Attraversa Sdo $n+1$ Stati ma se $m \geq k = n$ allora $m+1 \geq n$, Quindi Ripeto Almeno 1 Stato visto che di Stati diversi ne ho n . (CICLO su un nodo)

Uso DEL PUMPING LEMMA: Per dimostrare la non Regolarità altrimenti dovresti dimostrare che non $\exists$ un Automa (DIFFICILE).

Devo Negare un Pezzo Alla Volta del Lemma Originale

$\forall K \in \mathbb{N}. \exists z \in L. |z| \geq K. \forall uvw \in \Sigma^*. z = uvw \wedge |v| > 0 \wedge |uv| \leq K$
t.c. $\exists i \in \mathbb{N}. uv^i w \notin L$

$\{a^n b^n \mid n \in \mathbb{N}\}$

K Generico Senza Ipotesi

$\Uparrow$

In genere se mi Richiede di Ricordarmi **INFORMAZIONI PRECEDENTI**,
Quindi MEMORIA allora non è Regolare.

Ricordarsi Le IPOTESI Con le Quali Si parte per la DIMOSTRAZIONE

DEVO RICORDARMI che z DEVE Appartenere al Linguaggio e non devo far sì che la mia Scelta IMPONGA VINCOLI su K che deve rimanere Generico.

PROPRIETÀ DI CHIUSURA LINGUAGGI REGOLARI: Sono Chiusi

Rispetto Alle 3 Operazioni: $L_1^* \text{ o } L_2^*$

$L_1 \cdot L_2$

$L_1 \cup L_2$

} Utili Per Esercizi per
SPLITTARE L e Semplificare
la Dimostrazione

UNIONE DEVE ESSERE FINITA

ALTRIMENTI non Ho NESSUNA CERTEZZA

Ricordarsi che **Linguaggio finito $\equiv$ Linguaggio Regolare.**

Sono chiusi Anche Rispetto ALL' Operazione di Complemento

NEGAZIONE DELL' INSIEME ←

Tutto Uguale Ma Prendo il COMPLEMENTO degli Stati finali $\equiv \overline{F}$

Linguaggi Chiusi Anche Rispetto ALL' Intersezione Finita $L_1 \cap L_2$
che Vale a dire $\overline{\overline{L_1} \cup \overline{L_2}}$

QUALI SONO LE DOMANDE DECIDIBILI?

→ Algoritmico
Terminante

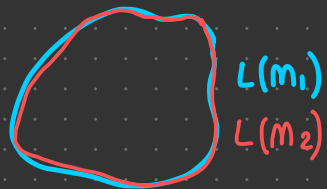
$L = L(M)$, M DFA con n Stati:

① $L \neq \emptyset$ sse Accetto una Stringa di lunghezza inferiore ad n

Dimostrare ESISTENZA non è decidibile ma con un Vincolo Sulle
lunghezza Imposto un limite e Prima o Poi Termina, Sempre Possibile

② L è Infinito sse M Accetta una Stringa di lunghezza L t.c.
 $n \leq L \leq 2n$

Dati 2 DFA, M_1 e M_2 , Stabilire che $L(M_1) = L(M_2)$ è DECIDIBILE
perchè $(L(M_1) \cap \overline{L(M_2)}) \cup (\overline{L(M_1)} \cap L(M_2)) = \emptyset$



GRAMMATICHE C.F.:

→ CONTEXT FREE → Libere dal Contesto

Per Linguaggi non Regolari... è una Quadrupla $\langle V, T, P, S \rangle$

Insieme finito di Simboli non Terminali

Insieme finito di Simboli Terminali

Insieme di Produzioni

Da Sottolineare che $V \cap T = \emptyset$

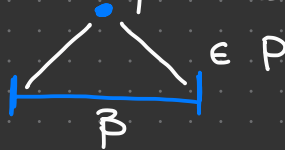
Regole di SOSTITUZIONI

$A \rightarrow \alpha$ dove $A \in V$ e $\alpha \in (V \cup T)^*$ [α Qualsiasi Seq. di Simboli]

$S \in V$ è il SIMBOLO INIZIALE

LINGUAGGIO GENERATO DA G:

Produzione $A \rightarrow \beta$ mi dice che A può Essere Sostituito con β in Qualsiasi Contesto



Posso Avere Anche più Passi di derivazione dove Ogni Passo è una DERIVAZIONE IMMEDIATA.

$$\alpha \Rightarrow_i \beta \equiv \exists i \in \mathbb{N}. \alpha \Rightarrow_i \beta$$

Dato $G = \langle T, P, S \rangle$ C.F. Il Linguaggio Generato

$$L(G) = \{x \in T^* \mid S \xRightarrow{*} x\}$$

Ovvero Sequenze di Simboli Terminali che posso Generare dal Simbolo Iniziale S .

Se ho Solo Simboli Terminali Allora Sono nel Linguaggio Generato

LINGUAGGIO CONTEXT-FREE $\Rightarrow L \subseteq T^*$ è C.F. se $\exists$ una Grammatica C.F. che lo Genera, Ovvero $L = L(G)$.

G_1 è Equivalente a G_2 se Generano Lo Stesso Linguaggio $L(G_1) = L(G_2)$

Come dimostriamo che L è Generato dalla Grammatica?

$$L = L(G) \text{ Vale a dire } L \subseteq L(G) \wedge L(G) \subseteq L$$

$$x \in L \Rightarrow S \xRightarrow{*} x$$

Per Induzione su $|x|$

$$S \rightarrow_i x \text{ allora } x \in L$$

Per Induzione su i , ovvero la lunghezza della derivazione

① Dimostro $L \subseteq L(G)$ Per Induzione Su $|x|$

BASE $\Rightarrow$ Prendo Stringa più Corta in L e Costruisco la Derivazione in G

IPOTESI $\Rightarrow$ Suppongo Che $\forall x \in L. |x| \leq n$ esiste $S \Rightarrow_* x$ e
Dobbiamo dimostrare che se $x \in L. |x| = n+1$ allora Esiste $S \Rightarrow_* x$

PASSO $\Rightarrow$ Riscriviamo x in Termini di x' t.c. $|x'| < |x|$ così su x' possiamo Applicare l'Ipotesi Induttiva.

Possiamo Poi Caratterizzare la derivazione per x' in G .

A Questo Punto da $S \Rightarrow_* x'$ dobbiamo ricostruire per x ragionando su G .

NB! Solo dopo che ho Applicato l'Ipotesi Induttiva ho ragionato su G .

② Dimostro $L(G) \subseteq L$ per Induzione Su i

Devo dimostrare che Tutte le Stringhe che Genero Stanno Nel mio Linguaggio e lo feci Sulla Lunghezza della DERIVAZIONE.

BASE $\Rightarrow$ Prendo derivazione di Simboli TERMINALI più Corta Possibile in G e Verifico che Quello che Genero $\in L$.

IPOTESI $\Rightarrow$ Suppongo che $S \Rightarrow_i x$ allora $x \in L$, devo dimostrare che se $S \Rightarrow_{i+1}$ allora $x \in L$

Solo dopo Aver Applicato I.I. posso Regionare Su L.

PASSO $\Rightarrow$ Suppongo di Avere $S \Rightarrow_{i+1} x$ e devo generare da Questa derivazione una derivazione lunga i per Applicare l'I.I.

$$S \Rightarrow_{i+1} x \equiv S \Rightarrow_i x_1 S x_2 \xrightarrow{i+1} x_1 x_2 = x$$

Una Volta Ottenuta La DERIVAZIONE Posso Applicare l'Ipotesi Induttiva e Spostare il Regionamento su L.

ALBERTI DI DERIVAZIONE: (Legati AL PARSING TREE)

Data $G = \langle V, T, P, S \rangle$ C.F., un Albero di DERIVAZIONE per G è definito Come Segue:

❧ Ogni Nodo ha un Etichetta in $V \cup T \cup \{ \epsilon \}$

❧ Radice ha Etichetta in V ($S \in V$)

❧ Nodo Interno (non Foglia) ha Etichetta in V .

❧ Se un Vertice N è Etichettato con A e $n_1, n_2, \dots, n_K$ Sono i figli di n e Sono Etichettati con $x_1, \dots, x_K \in V \cup T \cup \{ \epsilon \}$



Allora $A \rightarrow x_1 x_2 \dots x_K \in P$ dove $K \geq 2$

❧ Se un Nodo ha Etichetta ϵ allora N è foglia ed è l'unico figlio del Vertice Padre.

❧ A è ETICHETTA di n e x è Etichetta di M_1

m_1 figlio di n se $A \rightarrow X \in P \quad x \in V \cup T$

In Generale mi Serve Per Rappresentare una DERIVAZIONE di una Stringa All' Interno di una Grammatica.

AMBIGUITÀ DI UNA GRAMMATICA: Se G mi Permette di Avere alberi $\neq$ a Seconda dell' Ordine di Applicazione delle Regole allora ho una GRAMMATICA AMBIGUA.

È detta Ambigua Quindi Se $\exists$ una Stringa per la Quale ho più di un Albero di derivazione

Linguaggio INERENTEMENTE AMBIGUO se $\forall G$ C.F. $L = L(G)$ dove G è Ambigua

SEMPLIFICAZIONE GRAMMATICA: (Come Minimizzare Automi)

Si Parla di FORME NORMALI Per le Grammatiche.

Essere C.F. fissa un NON Terminale a SX e una Generica Seq. in $V \cup T$ a dx.

dove Vengono Applicati Vincoli Per le Forme Normali

FORMA NORMALE DI CHOMSKY $\Rightarrow A \rightarrow BC \vee A \rightarrow a$ dove $a \in T$ e $A, B, C \in V$

Quindi Albero di derivazione è BINARIO.

FORMA DI GREIBACH $\Rightarrow A \rightarrow aX$ dove $A \in T$ e $X \in (V \cup T)^*$

FORMA NORMALE DI CHOMSKY: Semplificazioni da fare:

- ❖ Deve $\exists$ derivazione che Coinvolga ogni Simbolo Terminale e non e ciò mi Porta ad un Eliminazione di Simboli Inutili
 - ❖ Non devono Esserci E-Produzioni ($A \rightarrow \epsilon$) ma Se $\epsilon \in L$ Solo il Simbolo iniziale può Averla ($S \rightarrow \epsilon$)
 - ❖ No produzioni Unitarie ($A \rightarrow B$) con $A, B \in V$
-

SIMBOLI UTILI: $x \in V \cup T$ è utile se Esiste Almeno una derivaz.

$$S \Rightarrow_* \alpha x \beta \Rightarrow_* w \quad w \in T^*$$

raggiungibile
da S

Deve Essere Coinvolto

Nella Generazione di Seq. Terminali

Quindi ad Esempio $S \rightarrow \alpha A \beta$ e $A \rightarrow A$ è INUTILE

① Eliminazione $x \in V$ che non Raggiungono $w \in T^*$

Data G , $L(G) \neq \emptyset$ si può Costruire una Grammatica $G = \langle V', T, P', S \rangle$ tale che $\forall A \in V'. \exists w \in T^*. A \Rightarrow_* w$

Insieme $T(\emptyset) =$ Insieme di non Terminali che in un Passo Vanno in un Terminale

e Posso Riapplicarla Per più Passi fino a che Vedo che non ho Nessun Aggiornamento.

Essendo $|V|$ finito So che la Procedura Termina in #Passi finito

② Simboli in $V \cup T$ non Raggiunti da S definisco:

$$T: P(V \cup T) \rightarrow P(V \cup T)$$

Quindi $T(\{S\}) =$ Insieme dei Simboli Raggiunti da S

$T(T(\{S\})) =$ Insieme di Simboli Raggiunti in una Produzione da S oppure da Simboli Raggiunti da S

Posso Smetterla di Applicare i PASSI Quando non ci Sono Più Aggiornamenti di Aggiunta All' Insieme.

PUMPING LEMMA PER C.F.:

Se L è un linguaggio CF Allora: dove $x, y, u, v, w \in T^*$

Esiste $K \in \mathbb{N}$ ($K \neq \emptyset$)

$\forall z \in L. |z| \geq K$

$\exists$ una Suddivisione di z in 5 Sottostringhe $z = uvwx^iy$ tali che $|vwx| \leq K$ e $|vx| > 0$

t.c. $\forall i \in \mathbb{N} \quad uv^iwx^iy \in L$

Schema è lo Stesso dei Regolari Ma leggermente più Complesso

Nel Caso in Cui Negassi Ho Mostrato che $L \notin C.F.$

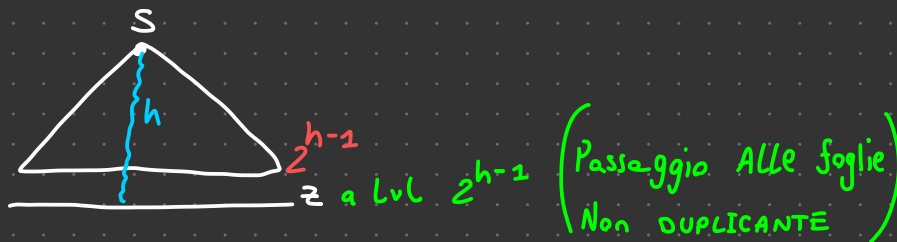
DIMOSTRAZIONE PUMPING LEMMA:

Se L è CF allora $\exists$ grammatica G tale che $L = L(G)$

$n = |V|$ (Simboli non Terminali)

Suppongo che G sia in forma Normale di Chomsky (SEMPRE POSSIBILE)

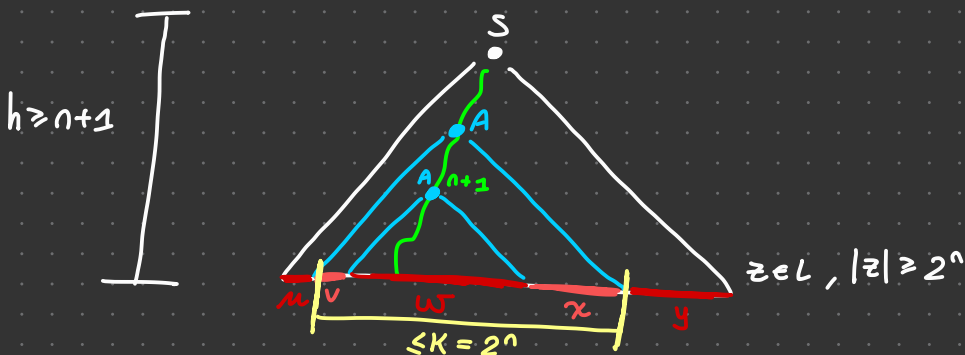
So Allora: $\forall z \in L$. Albero di derivazione Per z è Binario



Prendo come $K = 2^n$ (# Nodi/Simboli Terminali).

$z \in L$ dove $|z| \geq 2^n = K$

L'Albero di derivazione Per z ha Altezza $\geq n+1 \Rightarrow \exists$ Almeno un Cammino lungo $n+1$, ma anche uno che Attraversa $n+2$ Nodi ed $n+1$ Nodi Interni ma noi ne Abbiamo Solamente n di Nodi distinti $\Rightarrow$ C'è un ciclo/Una Ripetizione



Vale se prendiamo Primo Simbolo che si Ripete $\Rightarrow |vwx| \leq 2^n = K$

Posso Aggiungere Quante Voglio la derivazione di A e Rimarrò Sempre Nel Linguaggio

PROPRIETÀ DI CHIUSURA: Operazioni che se Applicate ad $L \in CF$ fanno Rimanere $L \in CF$.

$$\left. \begin{array}{l} \text{/// } L_1 \cup L_2 \\ \text{/// } L_1 \cdot L_2 \\ \text{/// } L_1^* \circ L_2^* \end{array} \right\} \in C.F.$$

Linguaggio Regolare $\in C.F.$ Visto che $C.F.$ è più Grande.

I $C.F.$ NON Sono Chiusi per INTERSEZIONE e COMPLEMENTO

PROPRIETÀ DI DECISIONE / ALGORITMI DI DECISIONE:

Decidibili : Seguenti Quesiti:

/// $L(G) = \emptyset$

/// $L(G)$ è finito

/// $L(G)$ è Infinito

/// $z \in L(G)$

Un Linguaggio REGOLARE è
CF

Nei linguaggi CF NON è decidibile L'EQUIVALENZA.

GERARCHIA DI CHOMSKY:

Grammatiche CS NON hanno Algoritmi Efficienti.

AUTOMI A PILA: Macchina per RICONOSCERE Linguaggi C.F.

Automati a Stati finiti + Memoria LIFO... Quando ha la Memoria

il NON DETERMINISMO Aumenta il Potere Espressivo.

$$M = \langle Q, \Sigma, R, \delta, q_0, z_0, F \rangle$$

Stati finiti $\leftarrow$ Q
Alfabeto $\leftarrow$ Σ
Alfabeto Pila $\leftarrow$ R
Simbolo Iniziale sulla PILA $\leftarrow$ z_0
Stato Iniziale $\leftarrow$ q_0

$$\delta: Q \times \underbrace{(\Sigma \cup \{\epsilon\})}_{\text{Simbolo Sul Nastro}} \times \underbrace{R}_{\text{Simbolo in cima Allo Stack (POP)}} \rightarrow P(Q \times R^*)$$

$\rightarrow$ Nuovo Stato

Push sullo Stack di una Seq. di Simboli

Descrizione Istantanea = $\langle q, x, \gamma \rangle$

$$x \in \Sigma^*$$

Simboli da leggere sul Nastro

$$\gamma \in R$$

Simboli Nella Pila

RICONOSCIMENTO PER STATO FINALE

$$L_F(M) = \{x \in \Sigma^* \mid (q_0, x, z_0) \xrightarrow{*} (q, \epsilon, \gamma) \mid q \in F, \gamma \in R^*\}$$

RICONOSCIMENTO PER PILA VUOTA

$$L_P(M) = \{x \in \Sigma^* \mid (q_0, x, z_0) \xrightarrow{*} (q, \epsilon, \epsilon) \mid q \in F\}$$

$F = \emptyset$

Stringhe Palindrome Con Separatore e CF ma Se non Avessi il Separatore Allora Necissiterei il non determinismo Perchè non so Quando Sono a Metà della Stringa.

$L = L(m)$ M automa ND con 1 Stato allora L è C.F.

Quindi Riduco gli Stati aumentando il Numero di Simboli (che è la VIRTUALIZZAZIONE DELLA MACCHINA)

→ Macchina Sorgente non Cambia Stato ma i Ambienti Avvengono in Memoria.

GERARCHIA DI CHOMSKY:

